DATA STRUCTURES AND ALGORITHMS LAB

(Course Code: 23UPCSC1L04)

A programming laboratory record submitted to Periyar University, Salem

In partial fulfillment of the requirements for the degree of

MASTER OF COMPUTER APPLICATIONS

By

KAYALVIZHI N

[Reg. No.: U24PG507CAP010]

DEPARTMENT OF COMPUTER SCIENCE

PERIYAR UNIVERSITY

(NAAC `A++` Grade with CGPA 3.61) – NIRF RANK 56 – ARIIA RANK 10

PERIYAR PALKALAI NAGAR,

SALEM – 636 011.

(APRIL - 2025)

CERTIFICATE

This is to certify that the Programming Laboratory entitled "**DATA STRUCTURE AND ALGORITHMS LAB (23UPCSC1L04)"** is a bonafide record work done by Mr. /Ms. _____ Register No: _____ as partial fulfillment of the requirements for the degree of Master of Computer Applications, in the Department of Computer Science, Periyar University, Salem, during the Academic Year 2024-2025.

Staff In-charge Head of the Department

Submitted for the practical examination held on……...……

Internal Examiner External Examiner

CONTENTS

S.NO	DATE	TITLE OF THE PROGRAM	PAGE NO	SIGNATURE
1.		Recursion concepts i) Linear recursion ii) Binary recursion		
2.		Stack ADT		
3.		Queue ADT		
4.		Doubly Linked List ADT		
5.		Heaps using Priority Queues		
6.		Merge sort		
7.		Quick sort		
8.		Binary Search Tree		
9.		Minimum Spanning Tree		
10.		Depth First Search Tree Traversal		

SOURCE CODE:

i) Linear Recursion:

```
def linear(list0, value, index):
if index >= len(list0):
    return "Not in"

if list0[index] == value:
    return index
return linear(list0, value, index + 1)

list1 = []
inputVal = int(input("Enter the range of the list: "))
for a in range(1, inputVal + 1):
    val = int(input(f"value-{a}: "))
    list1.append(val)

print(f"The list value is {list1}")
x = int(input("Enter the search Element: "))
i = 0
print(f"{x} value is present in index: {linear(list1, x, i)}")
```

<u>OUTPUT</u> (Linear Recursion) **:**

```
Enter the range of the list:  6
value-1:  2
value-2:  8
value-3:  0
value-4:  4
value-5:  1
value-6:  3
The list value is [2, 8, 0, 4, 1, 3]
Enter the search Element:  4
4 value is present in index: 3
```

SOURCE CODE:

ii) Binary Recursion:

```
def fib(n):
    if n <= 1:
        return n
    else:
        return fib(n-1)+fib(n-2)
num = int(input("Enter the Number want to generate fibonacci series:"))
for i in range(num):
    print(fib(i), end=" ")
```

<u>OUTPUT</u> (Binary Recursion)**:**

```
Enter the Number want to generate fibonacci series: 6
0 1 1 2 3 5
```

SOURCE CODE:

```
class Stack:
def __init__(self,size):
    self.arr = [None]*size
    self.capacity = size
    self.top =- 1

def push(self,x):
    if self.isFull():
        print("Stack overflow!!Calling Exit()....")
        exit(1)
    print(f"Inserting {x} into the stack....")
    self.top += 1
    self.arr[self.top] = x

def pop(self):
    if self.isEmpty():
        print("Stack underflow!!.Calling exit()....")
        exit(1)
    print(f"Removing {self.peek()} from the stack")
    top = self.arr[self.top]
    self.top -= 1
    return top

def peek(self):
     if self.isEmpty():
         exit(1)
     return self.arr[self.top]

def size(self):
    return self.top+1
```

```
    def isEmpty(self):
        return self.size() == 0

    def isFull(self):
        return self.size() == self.capacity

def main_stack():
    stack_size = int(input("Enter the size of the stack:"))
    stack = Stack(stack_size)

    print("\nMenu")
    print("1.Push the element to the stack")
    print("2.pop the element from the stack")
    print("3.Top element")
    print("4.Check if the stack is empty")
    print("5.Check if the stack is full")
    print("6.Exit")

    while True:
        choice = int(input("Enter your choice:"))

        if choice == 1:
            element = int(input("Enter the element to push:"))
            stack.push(element)
        elif choice == 2:
            popped_element = stack.pop()
            print(f"Popped element: {popped_element}")
        elif choice == 3:
            print("Top element:",stack.peek())
        elif choice == 4:
            print("Is the stack is empty?",stack.isEmpty())
        elif choice == 5:
            print("Is the stack is full?",stack.isFull())
```

```
        elif choice == 6:
            print("Exiting the program")
            break
        else:
            print("Invalid choice.Please enter the valid option")

if __name__ == '__main__':
    main_stack()
```

OUTPUT:

```
Enter the size of the stack: 5

Menu
1.Push the element to the stack
2.pop the element from the stack
3.Top element
4.Check if the stack is empty
5.Check if the stack is full
6.Exit
Enter your choice: 1
Enter the element to push: 2
Inserting 2 into the stack....
Enter your choice: 1
Enter the element to push: 4
Inserting 4 into the stack....
Enter your choice: 1
Enter the element to push: 6
Inserting 6 into the stack....
Enter your choice: 2
Removing 6 from the stack
Popped element: 6
Enter your choice: 3
Top element: 4
Enter your choice: 4
Is the stack is empty? False
Enter your choice: 5
Is the stack is full? False
Enter your choice: 6
Exiting the program
```

SOURCE CODE:

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        else:
            print("Queue is empty.")
            return None

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

    def front(self):
        if not self.is_empty():
            return self.items[0]
        else:
            print("Queue is empty.")
            return None

    def rear(self):
        if not self.is_empty():
            return self.items[-1]
        else:
```

```
        print("Queue is empty.")
        return None

    def display(self):
        print("Queue:", self.items)


def main():
    queue = Queue()

    print("\nMenu:")
    print("1. Enqueue element")
    print("2. Dequeue element")
    print("3. Check if the queue is empty")
    print("4. Display queue size")
    print("5. Display front element")
    print("6. Display rear element")
    print("7. Display queue")
    print("8. Exit")

    while True:
        choice = int(input("Enter your choice: "))

        if choice == 1:
            item = input("Enter the element to enqueue: ")
            queue.enqueue(item)
            print(f"{item} enqueued.")
        elif choice == 2:
            dequeued_item = queue.dequeue()
            if dequeued_item is not None:
                print(f"{dequeued_item} dequeued.")
        elif choice == 3:
            print("Is the queue empty?", queue.is_empty())
```

```
        elif choice == 4:
            print("Queue size:", queue.size())
        elif choice == 5:
            front_element = queue.front()
            if front_element is not None:
                print("Front element:", front_element)
        elif choice == 6:
            rear_element = queue.rear()
            if rear_element is not None:
                print("Rear element:", rear_element)
        elif choice == 7:
            queue.display()
        elif choice == 8:
            print("Exiting the program.")
            break
        else:
            print("Invalid choice. Please enter a valid option.")


if __name__ == '__main__':
    main()
```

OUTPUT:

```
Menu:
1. Enqueue element
2. Dequeue element
3. Check if the queue is empty
4. Display queue size
5. Display front element
6. Display rear element
7. Display queue
8. Exit
Enter your choice:  1
Enter the element to enqueue:  2
2 enqueued.
Enter your choice:  1
Enter the element to enqueue:  4
4 enqueued.
Enter your choice:  1
Enter the element to enqueue:  8
8 enqueued.
Enter your choice:  2
2 dequeued.
Enter your choice:  3
Is the queue empty? False
Enter your choice:  4
Queue size: 2
Enter your choice:  5
Front element: 4
Enter your choice:  6
Rear element: 8
Enter your choice:  7
Queue: ['4', '8']
Enter your choice:  8
Exiting the program.
```

SOURCE CODE:

```
import os

class _Node:
    __slots__ = '_element', '_link', '_prev'

    def __init__(self, element, link, prev):
        self._element = element
        self._link = link
        self._prev = prev

class DoublyLL:
    def __init__(self):
        self._head = None
        self._tail = None
        self._size = 0

    def __len__(self):
        return self._size

    def isempty(self):
        return self._size == 0

    def addLast(self, e):
        newest = _Node(e, None, None)
        if self.isempty():
            self._head = newest
        else:
            self._tail._link = newest
            newest._prev = self._tail
        self._tail = newest
        self._size += 1
```

```
    def addFirst(self, e):
        newest = _Node(e, None, None)
        if self.isempty():
            self._head = newest
            self._tail = newest
        else:
            newest._link = self._head
            self._head._prev = newest
            self._head = newest
        self._size += 1

    def addAnywhere(self, e, index):
        if index > self._size:
            print(f'Index value out of range, it should be between 0 -
{self._size}')
        elif index == self._size:
             self.addLast(e)
        elif index == 0:
             self.addFirst(e)
        else:
            newest = _Node(e, None, None)
            p = self._head
            for _ in range(index - 1):
                p = p._link
            newest._link = p._link
            p._link._prev = newest
            newest._prev = p
            p._link = newest
            self._size += 1

    def removeFirst(self):
        if self.isempty():
```

```
            print('List is already empty')
            return
        e = self._head._element
        self._head = self._head._link
        self._size -= 1
        if self.isempty():
            self._tail = None
        else:
            self._head._prev = None
        return e

    def removeLast(self):
        if self.isempty():
            print("List is already empty")
            return
        e = self._tail._element
        self._tail = self._tail._prev
        self._size -= 1
        if self.isempty():
            self._head = None
        else:
            self._tail._link = None
        return e

    def removeAnywhere(self, index):
        if index >= self._size:
            print(f'Index value out of range, it should be between 0 -
{self._size - 1}')
        elif self.isempty():
            print("List is empty")
        elif index == 0:
            return self.removeFirst()
        elif index == self._size - 1:
```

```
            return self.removeLast()
        else:
            p = self._head
            for _ in range(index - 1):
                p = p._link
            e = p._link._element
            p._link = p._link._link
            p._link._prev = p
            self._size -= 1
            return e

    def display(self):
        if self.isempty():
            print("List is Empty")
            return
        p = self._head
        print("NULL<-->", end=")
        while p:
            print(p._element, end="<-->")
            p = p._link
        print("NULL")
        print(f"\nHead : {self._head._element}, Tail :
{self._tail._element}")

def options():
    options_list = ['Add Last', 'Add First', 'Add Anywhere', 'Remove
First', 'Remove Last', 'Remove Anywhere', 'Display List', 'Exit']
    print("MENU")
    for i, option in enumerate(options_list):
        print(f'{i + 1}. {option}')
    choice = int(input("Enter choice: "))
    return choice
```

```
def switch_case(choice):
    os.system('cls')
    if choice == 1:
        elem = int(input("Enter Item: "))
        DL.addLast(elem)
        print("Added Item at Last!\n\n")
    elif choice == 2:
        elem = int(input("Enter Item: "))
        DL.addFirst(elem)
        print("Added Item at First!\n\n")
    elif choice == 3:
        elem = int(input("Enter Item: "))
        index = int(input("Enter Index: "))
        DL.addAnywhere(elem, index)
    elif choice == 4:
        print("Removed Element from First:", DL.removeFirst())
    elif choice == 5:
        print("Removed Element from last:", DL.removeLast())
    elif choice == 6:
        index = int(input("Enter Index: "))
        print(f"Removed Item: {DL.removeAnywhere(index)} !\n\n")
    elif choice == 7:
        print("List:")
        DL.display()
        print("\n")
    elif choice == 8:
        import sys
        sys.exit()

if __name__ == '__main__':
    DL = DoublyLL()
    while True:
        try:
```

```
        choice = options()
        switch_case(choice)
    except SystemExit:
        print("Exiting the program.")
        break
```

OUTPUT:

```
MENU
1. Add Last
2. Add First
3. Add Anywhere
4. Remove First
5. Remove Last
6. Remove Anywhere
7. Display List
8. Exit

Enter choice:  1
Enter Item:  2

Added Item at Last!


MENU
1. Add Last
2. Add First
3. Add Anywhere
4. Remove First
5. Remove Last
6. Remove Anywhere
7. Display List
8. Exit

Enter choice:  1
Enter Item:  4

Added Item at Last!


MENU
1. Add Last
2. Add First
3. Add Anywhere
4. Remove First
5. Remove Last
6. Remove Anywhere
7. Display List
8. Exit

Enter choice:  2
Enter Item:  6

Added Item at First!
```

```
MENU
1. Add Last
2. Add First
3. Add Anywhere
4. Remove First
5. Remove Last
6. Remove Anywhere
7. Display List
8. Exit

Enter choice:  2
Enter Item:  8

Added Item at First!


MENU
1. Add Last
2. Add First
3. Add Anywhere
4. Remove First
5. Remove Last
6. Remove Anywhere
7. Display List
8. Exit

Enter choice:  3
Enter Item:  10
Enter Index:  2

MENU
1. Add Last
2. Add First
3. Add Anywhere
4. Remove First
5. Remove Last
6. Remove Anywhere
7. Display List
8. Exit

Enter choice:  7

List:
NULL<-->8<-->6<-->10<-->2<-->4<-->NULL

Head : 8, Tail : 4
```

```
MENU
1. Add Last
2. Add First
3. Add Anywhere
4. Remove First
5. Remove Last
6. Remove Anywhere
7. Display List
8. Exit

Enter choice:  4

Removed Element from First: 8
MENU
1. Add Last
2. Add First
3. Add Anywhere
4. Remove First
5. Remove Last
6. Remove Anywhere
7. Display List
8. Exit

Enter choice:  5

Removed Element from last: 4
MENU
1. Add Last
2. Add First
3. Add Anywhere
4. Remove First
5. Remove Last
6. Remove Anywhere
7. Display List
8. Exit

Enter choice:  6
Enter Index:  2

Removed Item: 2 !
```

```
MENU
1. Add Last
2. Add First
3. Add Anywhere
4. Remove First
5. Remove Last
6. Remove Anywhere
7. Display List
8. Exit

Enter choice:  8

Exiting the program.
```

SOURCE CODE:

```
import heapq

class MaxHeap:
    def __init__(self):
        self._heap = []

    def insert(self,item):
        heapq.heappush(self._heap,-item)

    def delete(self):
        if len(self._heap) > 0:
            return -heapq.heappop(self._heap)
        else:
            raise Exception("Heap is Empty")

    def peek(self):
        if len(self._heap) > 0:
            return self._heap[0]
        else:
            raise Exception("Heap is Empty")

    def size(self):
        return len(self._heap)

class PriorityQueue:
    def __init__(self):
        self._heap = MaxHeap()

    def push(self,item,priority):
        self._heap.insert(priority)
```

```
    def pop(self):
        if self.is_empty():
            raise IndexError("Pop from an empty priority queue")
        return self._heap.delete()
    def is_empty(self):
        return self._heap.size() == 0

pq = PriorityQueue()

print("\nMENU")
print("1.Add task and priority")
print("2.Remove task with highest priority")
print("3.Exit")
while True:
    choice = input("Enter your choice:")
    if choice == '1':
        task = input("Enter the task:")
        priority = int(input("Enter the priority:"))
        pq.push(task, priority)
        print(f"Task '{task}' with priority {priority} added to the priority
queue")
    elif choice == '2':
        try:
            removed_task = pq.pop()
            print(f"Task: '{removed_task}' with highest priority removed
from the priority queue")
        except IndexError:
            print("Priority queue is empty cannot remove task")
    elif choice == '3':
        print("Exiting the program")
        break
    else:
        print("Invalid choice. Please enter the valid option.")
```

OUTPUT:

```
MENU
1.Add task and priority
2.Remove task with highest priority
3.Exit
Enter your choice: 1
Enter the task: 2
Enter the priority: 1
Task '2' with priority 1 added to the priority queue
Enter your choice: 1
Enter the task: 4
Enter the priority: 2
Task '4' with priority 2 added to the priority queue
Enter your choice: 2
Task: '2' with highest priority removed from the priority queue
Enter your choice: 3
Exiting the program
```

SOURCE CODE:

```
def merge_sort(arr):
    if len(arr) <=  1:
        return arr
    mid = len(arr)//2
    left = arr[:mid]
    right = arr[mid:]
    left = merge_sort(left)
    right = merge_sort(right)
    return merge(left,right)

def merge(left,right):
    result = []
    i,j = 0,0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i+=1
        else:
            result.append(right[j])
            j+=1
    result+=left[i:]
    result+=right[j:]
    return result

user_input=input("Enter the list of numbers separated by spaces:")
arr=[int(num) for num in user_input.split()]
sorted_arr=merge_sort(arr)
print("Input Array:",arr)
print("Sorted Array:",sorted_arr)
```

OUTPUT:

```
Enter the list of numbers separated by spaces: 31 12 14 28 17 19
Input Array: [31, 12, 14, 28, 17, 19]
Sorted Array: [12, 14, 17, 19, 28, 31]
```

SOURCE CODE:

```
def quick_sort(alist,start,end):
    if end-start>1:
        p = partition(alist,start,end)
        quick_sort(alist,start,p)
        quick_sort(alist,p+1,end)

def partition(alist,start,end):
    pivot = alist[start]
    i = start+1
    j = end-1
    while True:
        while i<=j and alist[i]<=pivot:
            i=i+1
        while i<=j and alist[j]>=pivot:
            j=j-1
        if i<=j:
            alist[i],alist[j]=alist[j],alist[i]
        else:
            alist[start],alist[j]=alist[j],alist[start]
            return j

alist=input("Enter the list of numbers:").split()
alist=[int(x) for x in alist]
quick_sort(alist ,0 ,len(alist) - 1)
print("Sorted List:",alist)
```

OUTPUT:

```
Enter the list of numbers: 28 14 17 31 12 19
Sorted List: [12, 14, 17, 28, 31, 19]
```

SOURCE CODE:

```
class binarySearchTree:
    def __init__(self, val=None):
        self.val = val
        self.left = None
        self.right = None

    def insert(self, val):
        if self.val is None:
            self.val = val
        else:
            if val == self.val:
                return 'No duplicates allowed in the binary search tree'
            if val < self.val:
                if self.left:
                    self.left.insert(val)
                else:
                    self.left = binarySearchTree(val)
            else:
                if self.right:
                    self.right.insert(val)
                else:
                    self.right = binarySearchTree(val)

    def breadthFirstSearch(self):
        currentNode = self
        bfs_list = []
        queue = []
        queue.insert(0, currentNode)
        while len(queue) > 0:
            currentNode = queue.pop()
            bfs_list.append(currentNode.val)
```

```
        if currentNode.left:
            queue.insert(0, currentNode.left)
        if currentNode.right:
            queue.insert(0, currentNode.right)
    return bfs_list

def depthFirstSearch_INorder(self):
    return self.traverseInOrder([])

def depthFirstSearch_PREorder(self):
    return self.traversePreOrder([])

def depthFirstSearch_POSTorder(self):
    return self.traversePostOrder([])

def traverseInOrder(self, lst):
    if self.left:
        self.left.traverseInOrder(lst)
    lst.append(self.val)
    if self.right:
        self.right.traverseInOrder(lst)
    return lst

def traversePreOrder(self, lst):
    lst.append(self.val)
    if self.left:
        self.left.traversePreOrder(lst)
    if self.right:
        self.right.traversePreOrder(lst)
    return lst

def traversePostOrder(self, lst):
    if self.left:
```

```
        self.left.traversePostOrder(lst)
    if self.right:
        self.right.traversePostOrder(lst)
    lst.append(self.val)
    return lst

def findNodeAndItsParent(self, val, parent=None):
    if val == self.val:
        return self, parent
    if val < self.val:
        if self.left:
            return self.left.findNodeAndItsParent(val, self)
        else:
            return 'Not found'
    else:
        if self.right:
            return self.right.findNodeAndItsParent(val, self)
        else:
            return 'Not found'

def delete(self, val):
    if self.findNodeAndItsParent(val) == 'Not found':
        return 'Node is not in tree'

    deleting_node, parent_node = self.findNodeAndItsParent(val)
    nodes_effected = deleting_node.traversePreOrder([])

    if len(nodes_effected) == 1:
        if parent_node is not None:
            if parent_node.left == deleting_node:
                parent_node.left = None
            else:
                parent_node.right = None
```

```
            else:
                self.val = None
                self.left = None
                self.right = None
        else:
            if parent_node is None:
                nodes_effected.remove(deleting_node.val)
                self.left = None
                self.right = None
                self.val = None
                for node in nodes_effected:
                    self.insert(node)
            else:
                nodes_effected.remove(deleting_node.val)
                if parent_node.left == deleting_node:
                    parent_node.left = None
                else:
                    parent_node.right = None

                if parent_node is not None:
                    nodes_effected.remove(parent_node.val)

                for node in nodes_effected:
                    self.insert(node)

        return 'Successfully deleted'

# Example usage with user input:
bst = binarySearchTree()

while True:
    print("\nMENU:")
    print("1. Insert Element")
```

```
        print("2. Breadth-First Search")
        print("3. Depth-First Search (In-Order)")
        print("4. Depth-First Search (Pre-Order)")
        print("5. Depth-First Search (Post-Order)")
        print("6. Delete Node")
        print("7. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            element = int(input("Enter the element to insert: "))
            print(bst.insert(element))
        elif choice == "2":
            print("Breadth-First Search:", bst.breadthFirstSearch())
        elif choice == "3":
            print("Depth-First Search (In-Order):",
bst.depthFirstSearch_INorder())
        elif choice == "4":
            print("Depth-First Search (Pre-Order):",
bst.depthFirstSearch_PREorder())
        elif choice == "5":
            print("Depth-First Search (Post-Order):",
bst.depthFirstSearch_POSTorder())
        elif choice == "6":
            node_to_delete = int(input("Enter the node value to delete: "))
            print(bst.delete(node_to_delete))
        elif choice == "7":
            print("Exiting the program.")
            break
        else:
            print("Invalid choice. Please enter a valid option.")
```

OUTPUT:

```
MENU:
1. Insert Element
2. Breadth-First Search
3. Depth-First Search (In-Order)
4. Depth-First Search (Pre-Order)
5. Depth-First Search (Post-Order)
6. Delete Node
7. Exit
Enter your choice:  1
Enter the element to insert:  4
Successfully inserted
Enter your choice:  1
Enter the element to insert:  5
Successfully inserted
Enter your choice:  1
Enter the element to insert:  6
Successfully inserted
Enter your choice:  1
Enter the element to insert:  7
Successfully inserted
Enter your choice:  1
Enter the element to insert:  8
Successfully inserted
Enter your choice:  2
Breadth-First Search: [4, 5, 6, 7, 8]
Enter your choice:  3
Depth-First Search (In-Order): [4, 5, 6, 7, 8]
Enter your choice:  4
Depth-First Search (Pre-Order): [4, 5, 6, 7, 8]
Enter your choice:  5
Depth-First Search (Post-Order): [8, 7, 6, 5, 4]
Enter your choice:  6
Enter the node value to delete:  4
Successfully deleted
Enter your choice:  7
Exiting the program.
```

<u>SOURCE CODE</u> (Prim's Algorithm) **:**

```
import heapq

class Graph:
    def __init__(self, vertices):
        self.V = vertices  # Number of vertices
        self.graph = {i: [] for i in range(vertices)}

    def add_edge(self, u, v, w):
        self.graph[u].append((w, v))
        self.graph[v].append((w, u))

    def prim_mst(self):
        min_heap = [(0, 0)]
        visited = set()
        mst = []
        total_weight = 0

        while len(visited) < self.V:
            weight, u = heapq.heappop(min_heap)
            if u in visited:
                continue
            visited.add(u)
            total_weight += weight

            if len(visited) > 1:
                mst.append((u, weight))

            for w, v in self.graph[u]:
                if v not in visited:
                    heapq.heappush(min_heap, (w, v))
```

```
        print("Edges in the Minimum Spanning Tree:")
        for u, w in mst:
            print(f"{u} -- {w}")
        print(f"Total weight of MST: {total_weight}")

vertices = int(input("Enter number of vertices: "))
edges = int(input("Enter number of edges: "))
g = Graph(vertices)

print("Enter the edges (u v w):")
for _ in range(edges):
    u, v, w = map(int, input().split())
    g.add_edge(u, v, w)

g.prim_mst()
```

<u>OUTPUT</u> (Prim's Algorithm)**:**

```
Enter number of vertices:  4
Enter number of edges:  5
Enter the edges (u v w):
 0 1 1
 0 2 3
 1 2 1
 1 3 4
 2 3 1
Edges in the Minimum Spanning Tree:
1 -- 1
2 -- 1
3 -- 1
Total weight of MST: 3
```

SOURCE CODE (Kruskal's Algorithm)**:**

```
class Graph:
    def __init__(self, vertices):
        self.V = vertices
        self.graph = []

    def add_edge(self, u, v, w):
        self.graph.append([u, v, w])

    def find(self, parent, i):
        if parent[i] == i:
            return i
        return self.find(parent, parent[i])

    def union(self, parent, rank, x, y):
        root_x = self.find(parent, x)
        root_y = self.find(parent, y)
        if rank[root_x] < rank[root_y]:
            parent[root_x] = root_y
        elif rank[root_x] > rank[root_y]:
            parent[root_y] = root_x
        else:
            parent[root_y] = root_x
            rank[root_x] += 1

    def kruskal_mst(self):
        self.graph = sorted(self.graph, key=lambda item: item[2])
        parent = []
        rank = []
        mst = []

        for node in range(self.V):
```

```
            parent.append(node)
            rank.append(0)

        edge_count = 0
        i = 0
        while edge_count < self.V - 1:
            u, v, w = self.graph[i]
            i += 1
            x = self.find(parent, u)
            y = self.find(parent, v)
            if x != y:
                mst.append([u, v, w])
                self.union(parent, rank, x, y)
                edge_count += 1

        print("Edges in the Minimum Spanning Tree:")
        for u, v, w in mst:
            print(f"{u} -- {v} == {w}")

vertices = int(input("Enter number of vertices: "))
edges = int(input("Enter number of edges: "))
g = Graph(vertices)

print("Enter the edges (u v w):")
for _ in range(edges):
    u, v, w = map(int, input().split())
    g.add_edge(u, v, w)

g.kruskal_mst()
```

<u>OUTPUT</u> (Kruskal's Algorithm)**:**

```
Enter number of vertices:  6
Enter number of edges:  9
Enter the edges (u v w):
 0 1 4
 0 2 3
 1 2 1
 1 3 2
 2 3 4
 3 4 2
 4 5 6
 2 5 5
 3 5 3
Edges in the Minimum Spanning Tree:
1 -- 2 == 1
1 -- 3 == 2
3 -- 4 == 2
0 -- 2 == 3
3 -- 5 == 3
```

SOURCE CODE:

```
from collections import defaultdict

class Graph:
    def __init__(self):
        self.graph = defaultdict(list)

    def add_edge(self, u, v):
        self.graph[u].append(v)

    def dfs_util(self, v, visited):
        visited[v] = True
        print(v, end=' ')
        for i in self.graph[v]:
            if not visited[i]:
                self.dfs_util(i, visited)

    def dfs(self, start_vertex):
        max_vertex = max(max(self.graph.keys()),
max(max(self.graph.values())))
        visited = [False] * (max_vertex + 1)
        self.dfs_util(start_vertex, visited)

num_vertices = int(input("Enter the number of vertices: "))
g = Graph()

num_edges = int(input("Enter the number of edges: "))
for _ in range(num_edges):
    u, v = map(int, input("Enter edge (u v): ").split())
    g.add_edge(u, v)
```

```
start_vertex = int(input("Enter the starting vertex for DFS: "))


print("Depth First Search (DFS) starting from vertex", start_vertex,
":")
g.dfs(start_vertex)
```

OUTPUT:

```
Enter the number of vertices:  6
Enter the number of edges:  5
Enter edge (u v):  1 3
Enter edge (u v):  3 4
Enter edge (u v):  4 5
Enter edge (u v):  5 6
Enter edge (u v):  6 2
Enter the starting vertex for DFS:  4
Depth First Search (DFS) starting from vertex 4 :
4 5 6 2
```